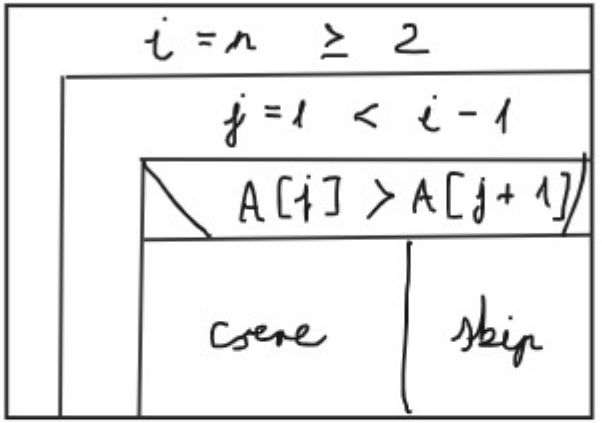
Algo vizsga gyakorlás

Bubble sort

Buborék(A/1:T[n])



Hányszor fut le (A.length = n)

n

$n + n-1 + n-2 \ldots + 2$

$n-1 + n-2 + \ldots + 1$

Cseresz´ám ?

Összehasonlítások száma:

$\frac{n^2 - n}{2} \in \Theta n^2$

miro

Javított bubble sort

2, 3, 1, 4, 5

2, 3, 1, 4, 5 $u = 2$

2, 1, 3, 4, 5

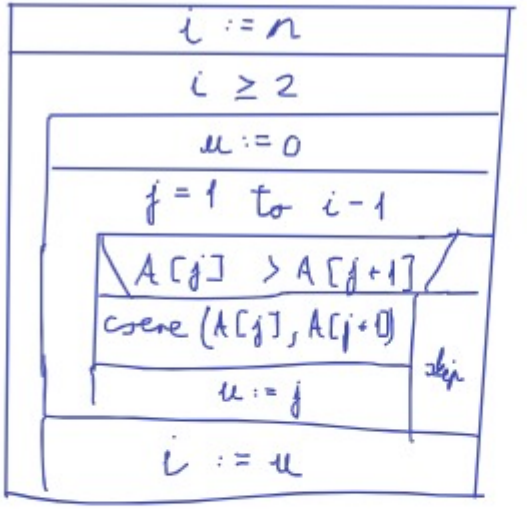
2, 1, 3, 4, 5

---

1. menet vége 3,4,5 rendezett, mivel $u = 2$ ezért az utolsó csere a 2. indexen volt

2, 1, 3, 4, 5

1, 2, 3, 4, 5 $u = 1$



miro

Maximum sort

4, 1, 5, 2, 3

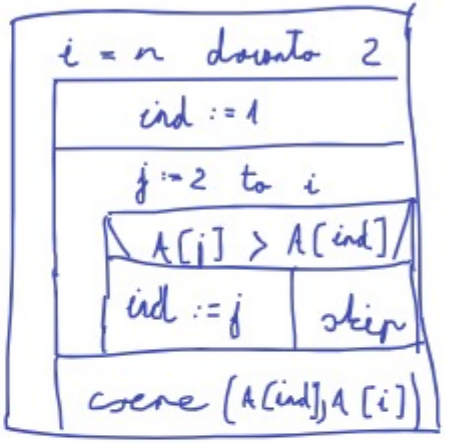
4, 1, 3, 2 | 5

2, 1, 3 | 4, 5

2, 1 | 3, 4, 5

1, 2, 3, 4, 5

● ← index
● ← maxelem

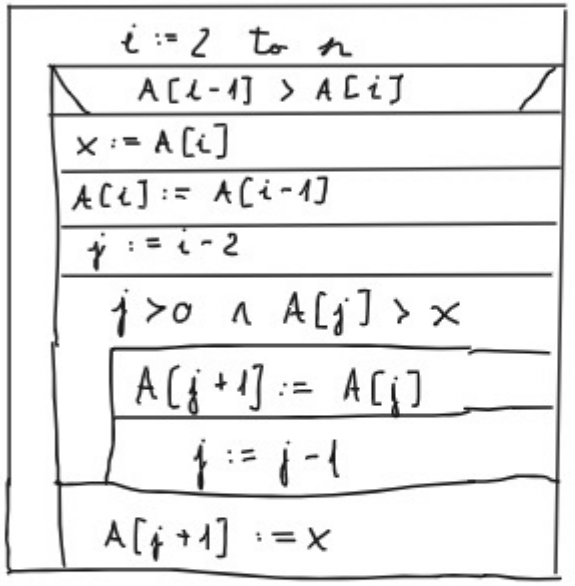


miro

Beszúró (Insertion) rendezés

$\Theta(n^2)$

insertionSort(A/1:T[n])



4, 3, 1, 2

3, 4, 1, 2

3, 4, 4, 2

1, 3, 4, 2

1, 3, 4, 4

1, 3, 3, 4

1, 2, 3 4

miro

Merge sort

(összefésülő rendezés)

$\Theta(n \cdot \log n)$

merge(A : T[n]; u, m, v : [0..n])

d := m - u
Z: T[d]; Z[0..d) := A[u..m)
k := u
j := 0; i := m
i < v and j < d

A[i] < Z[j]	
true	*false*
A[k] := A[i]	A[k] := Z[j]
i := i + 1	j := j + 1
k := k + 1	

j < d
A[k] := Z[j]
k := k + 1
j := j +1

sorted merge of A[u..m) and A[m..v) into A[u..v)

ms(A : T[n]; u, v : [0..n])

u < v - 1	
true	*false*
m := (u + v) / 2 // lefele kerekítve	
ms(A, u, m)	
ms(A, m, v)	
merge(A, u, m, v)	

(A[u..v))

mergesort(A : T[n])

ms(A, 0, n)

miro

$(a+b)\cdot(c-d)\,/\,f\,\wedge\,(g-h)+j-l-e$

lengyel forma

$a\ b\ +\ c\ d\ -\ *\ f\ g\ h\ -\ \wedge\ /\ j\ +\ l\ -\ e\ -$

$X = a\cdot b\wedge c + (d-e/f)\cdot(g+h)$

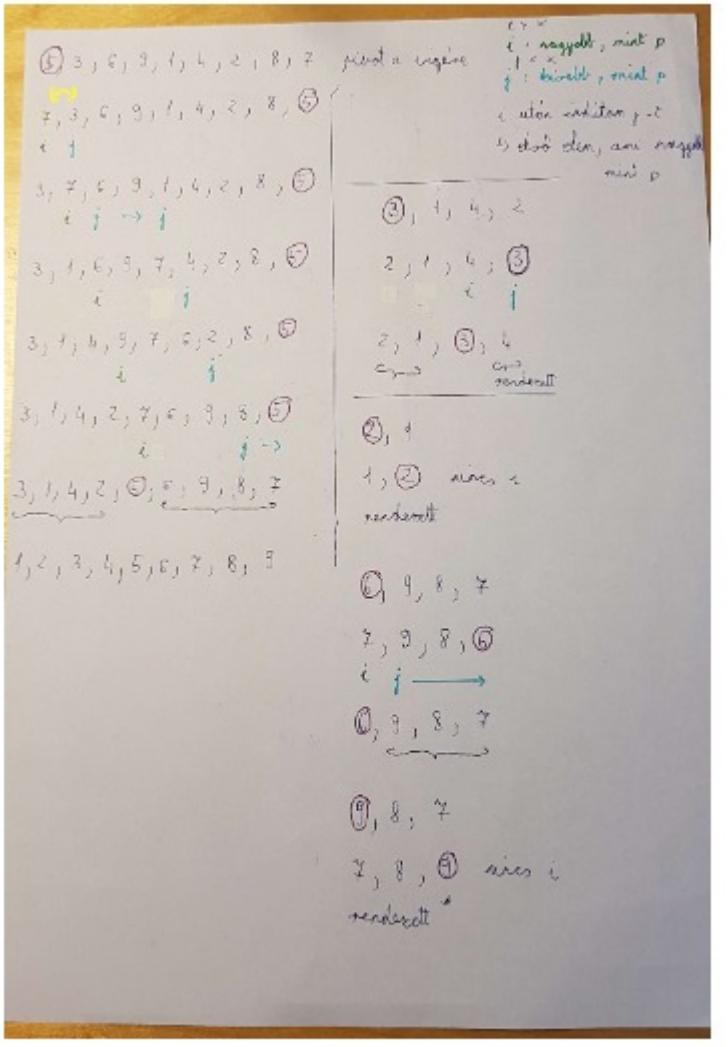
$X\ a\ b\ c\ \wedge\ *\ d\ e\ f\ /\ -\ g\ h\ +\ *\ +\ =$

miro

Quick sort $\Omega(n \cdot \log n)$

$O(n^2)$

nem stabil rendezés



miro

Láncolt lista

Rendezett esetben: beszúrás / törlés nem igényel elem mozgatást

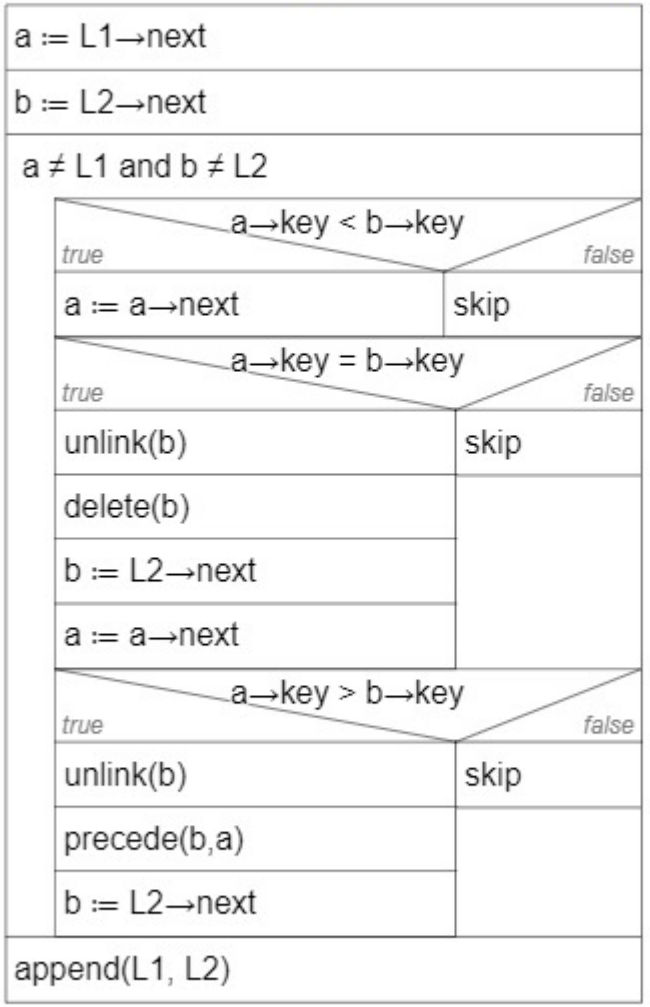
↳ hely megtalálása : $O(n)$

Nem indexelhető konstans műveletigénnyel

↳ $O(n)$

miro

union (L1, L2 : E2*)



Insert text here

miro

Radix sort

d: helyiértékek száma $\quad \Theta(d \cdot \text{rendezés}(n))$

r: számrendszer

helyiérték szerint rendezünk stabil rendezéssel

Counting sort

$\Theta(n+r)$

Bucket sort

$O(n \cdot \log n)$

$\Omega(n)$

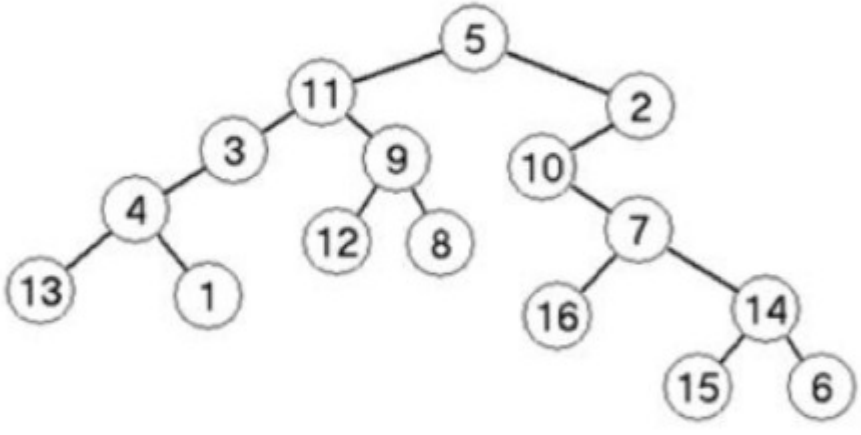
miro

Bináris fák

Bináris fa: Egy csúcsának egy bal és egy jobb gyereke lehet.

Csúcs: a fa egy adateleme
Root / gyökér: A kiinduló csúcs
Levél: Csúcs, amelynek nincs gyereke

Magasság: A fa szintjeinek száma (0-tól számolva)

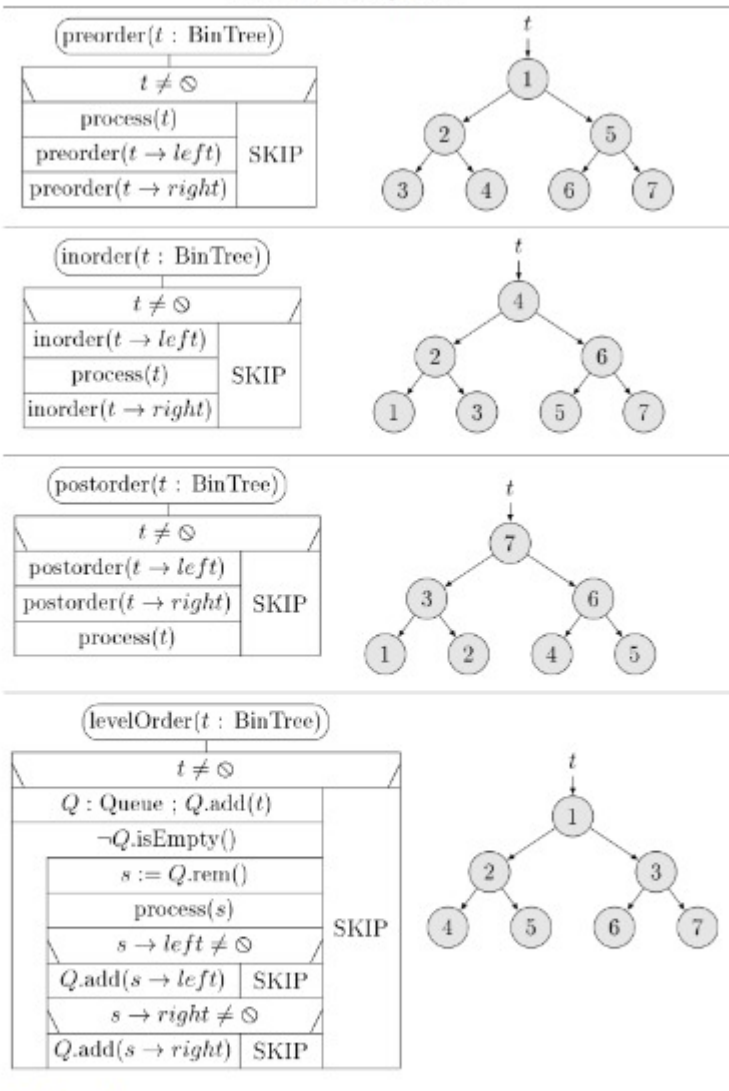


Preorder: 5, 11, 3, 4, 13, 1, 9, 12, 8, 2, 10, 7, 16, 14, 15, 6

Inorder: 13, 4, 1, 3, 11, 12, 9, 8, 5, 10, 16, 7, 15, 14, 6, 2

Postorder: 13, 1, 4, 3, 12, 8, 9, 11, 16, 15, 6, 14, 7, 10, 2, 5

Bináris fák bejárása



$\Theta(n)$

miro

Bináris fa fogalmak

Bináris fa: Minden csúcsnak egy bal és egy jobb gyereke lehet.

Bináris keresőfa:
A tetszőleges csúcs kulcsánál a bal részfájában minden csúcs kulcsa kisebb. A jobb részfában lévő kulcsok pedig nagyobban.

Bináris rendezőfa:
Olyan bináris keresőfa, amely megengedi az egyenlőséget a kulcsoknál.

Szigorúan bináris fa:
A fa minden belső pontjának pontosan 2 gyereke van.

Teljes bináris fa:
Olyan szigorúan bináris fa, ahol minden levél azonos szinten helyezkedik el.

Majdnem teljes bináris fa:
Olyan teljes bináris fa, amelynek alsó szintjéről elhagytunk néhány levelet (nem az összeset!).

Majdnem teljes balra tömörített bináris fa / szintfolytonos fa:
Olyan majdnem teljes bináris fa, amelynek a jobb oldaláról hiányoznak levelek.

miro

Kupac (heap)

Maximum kupac:
Olyan majdnem teljes, balra tömörített bináris fa, melynek minden belső pontjára teljesül, hogy a belső pont kulcsa nagyobb vagy egyenlő a gyerekei kulcsánál.
(A kupac gyökerében van a legnagyobb elem.)

Minimum kupac:
A szülő kulcs kisebb vagy egyenlő a gyerekei kulcsánál.

Reprezentáljuk a heap-et egy 1-től indexelt tömbbel

Ekkor egy adott csúcs indexe: i
Bal gyerek indexe: 2*i
Jobb gyerek indexe: 2*i+1
Szülő indexe: i/2 (lefele kerekítve)

miro

1. Algoritmusok és adatszerkezetek I. vizsga, 2023.05.30.

Struktogram készítésekor a feladat része (1) a paraméterek típusának megadása, (2) a cím szerinti paraméterátvétel szükség szerinti jelölése, (3) az esetleges visszatérési típus megadása, (4) *a láncolt adatszerkezetek elemtípusának (pl.* E1, E2, Node*) UML leírása.*

1.a, Mutassa be a *számjegypozíciós* (*Radix*) *rendezés* működését a következő, négyes számrendszerbeli számok tömbjén: $\langle 20, 02, 21, 01, 31, 20 \rangle$! Az egyes menetekben *leszámláló rendezést* alkalmazzon!
1.b, Mekkora a fenti rendezés aszimptotikus műveletigénye, és miért?
1.c, A *leszámláló rendezés* – mint segédprogram – mely tulajdonságára épül a *Radix rendezés*? Miért teljesül ez a tulajdonság az adott esetben, és hogyan biztosítja a Radix rendezés helyességét?

2.a, Adja meg a **mergeSort**$(A : \mathcal{T}[])$ és segédeljárásai struktogramjait!
2.b, Mekkora lesz a műveletigénye? Miért? (Elég vázlatosan indokolni.)

3.a, Mondja ki a bináris rendezőfa definícióját!
3.b, A t:Node* egy láncoltan ábrázolt bináris rendezőfát azonosít. **Írja meg** az insert(t, k) eljárás rekurzív, ciklust nem tartalmazó struktogramját, $MT(h) \in \Theta(h)$ műveletigénnyel! (Ez a beszúrás, tetszőleges k kulcsra pontosan egy csúccsal növeli a rendezőfa méretét.)
3.c, Hogyan tudna a fenti eljárásra stabil rendezést építeni?

4. A $T[0..(m-1)]$ hasító tábla rései kétirányú, nemciklikus, fejelem nélküli, rendezetlen láncolt listák pointerei. Adott a $k \bmod m$ hasító függvény. A kulcsütközéseket láncolással oldjuk fel. Mindegyik kulcs csak egyszer szerepelhet T-ben.
4.a, Írja meg a *search*(T, k) függvényt, ami visszaadja a T-beli, k kulcsú listaelem címét, vagy a $\oslash$ pointert, ha ilyen nincs!
4.b, Írja meg a *del*(T, p) eljárást, ami törli a T hasító táblából (és deallokálja is) a p pointer által mutatott listaelemét! $(p \neq \oslash)$
4.c, Mi a kitöltöttségi hányados?
4.d, Milyen aszimptotikus becslést tud adni a fenti műveletek minimális, átlagos és maximális futási idejére? Miért?

miro

Vizsga 05. 30.

1.a.) 20, 02, 21, 01, 31, 20

0	1	2	3	4	5
20	20	21	01	31	02

	20	02	21	01	31	20	C	C	20	31	01	21	02	20
0	+					+	2	2	1					0
1			+	+	+		3	5		4	3	2		
2		+					1	6					5	
3							0	6						

	20	20	21	01	31	02	C	C	02	31	01	21	20	20	
0				+		+	2	2	1		0				0
1							0	2							1
2	+	+	+				3	5				4	3	2	2
3					+		1	6		5					3

0 1 2 3 4 5

01 02 20 20 21 31

miro

1. b) $r = 4$ $\Theta(d \cdot \text{rendezés}(n))$

$d = 2$ counting : $\Theta(n)$

$\Theta(2 \cdot n)$ $2 \cdot 2 \cdot n \rightarrow n \Rightarrow MT(n) \in \Theta(n)$



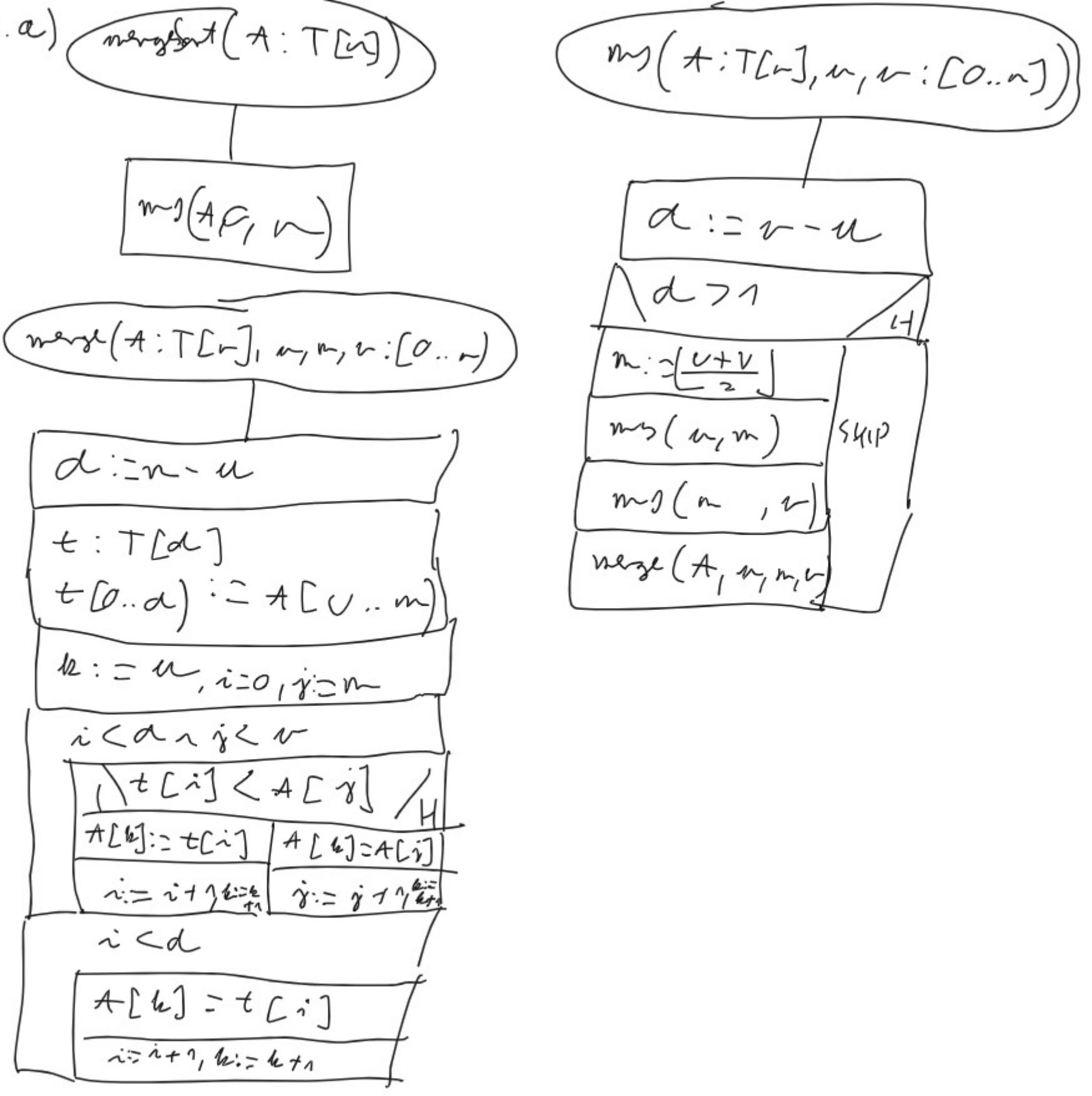
Amongus SUS! Sussy Baka

1. c)

A counting (leszámoló) rendezés egy stabil rendezés. Azaz az azonos kulcsú elemeket nem cseréli fel. Ez biztosítja a radix helyes működését.

miro

2. a)



Dobor edition merdzsözoltán

miro

Merge sort levezetve

2.a)

merge(A : T[n]; u, m, v : [0..n])

d := m - u
Z: T[d]; Z[0..d) := A[u..m)
k := u
j := 0; i := m
i < v and j < d

A[i] < Z[j]	
true	*false*
A[k] := A[i]	A[k] := Z[j]
i := i + 1	j := j + 1
k := k + 1	

j < d
A[k] := Z[j]
k := k + 1
j := j +1

sorted merge of A[u..m) and A[m..v) into A[u..v)

ms(A : T[n]; u, v : [0..n])

u < v - 1	
true	*false*
m := (u + v) / 2 // lefele kerekítve	
ms(A, u, m)	
ms(A, m, v)	
merge(A, u, m, v)	

(A[u..v))

mergesort(A : T[n])

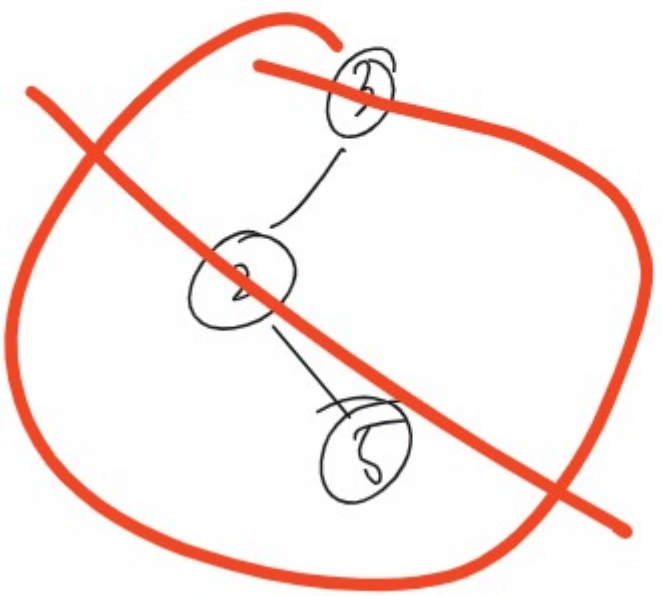
ms(A, 0, n)

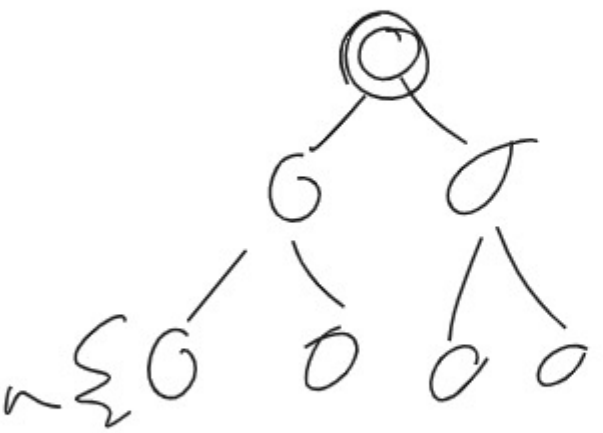
2.b) $\Theta(n \cdot \log n)$

1 merge $\Theta(v-u)$.

Ez ms felezi a tömböt, ezért kb. $\log n + 1$.

$\Theta(n \cdot \log n)$





$4 : 2^h = 1$

$4 = 2^h$

$h = \log_2(4)$

miro

3.a)

Bináris rendezőfa definíciója

Bináris fa: Minden csúcsnak egy bal és egy jobb gyereke lehet.

Bináris keresőfa:
A tetszőleges csúcs kulcsánál a bal részfájában minden csúcs kulcsa kisebb. A jobb részfában lévő kulcsok pedig nagyobban.

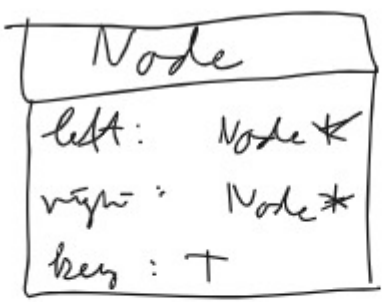
Bináris rendezőfa:
Olyan bináris keresőfa, amely megengedi az egyenlőséget a kulcsoknál.

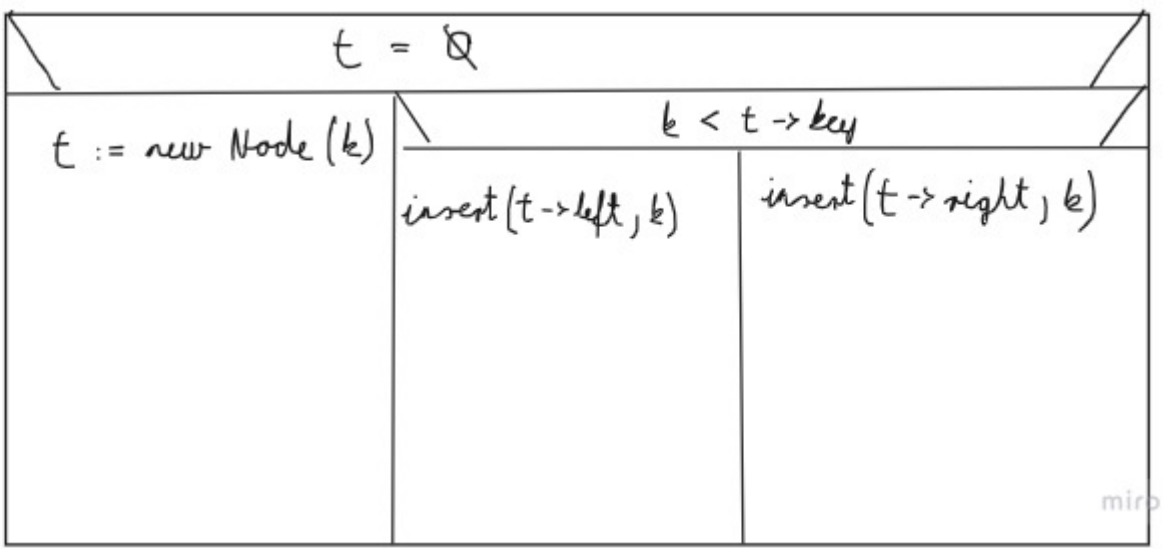
3.b)

$\Theta(h)$ t : Node* insert(t, k)

insert(&t : Node*; k : T)

\varnothing



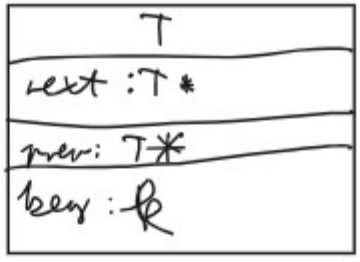


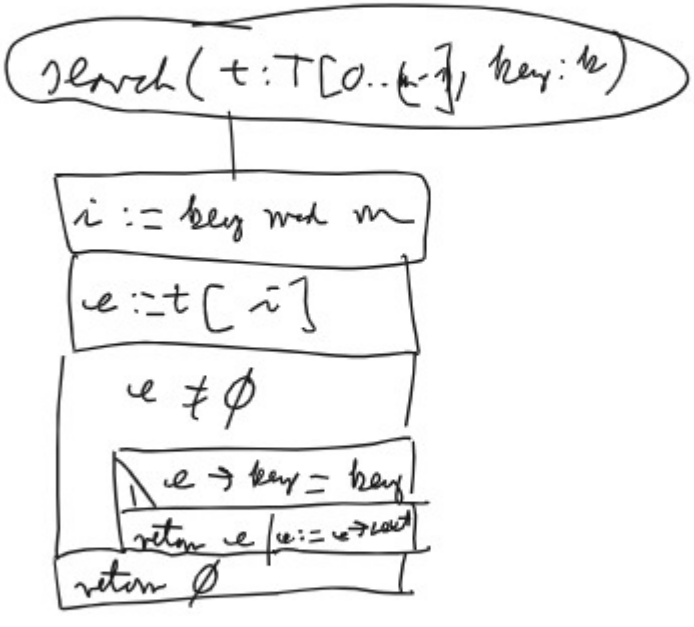
3.c)

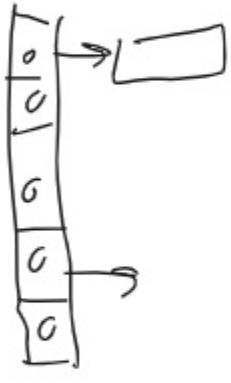
Használjuk ki a rendezőfa rendezettségét. Az inorder bejárás rendezve adja vissza a fa elemeit.
Az elemeket egy sorban (queue) tudjuk tárolni.

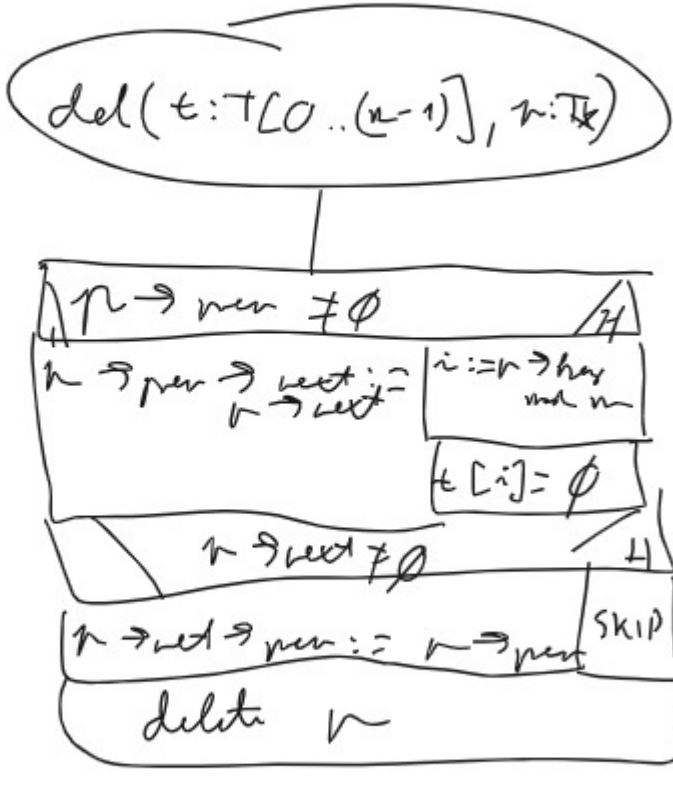
4.a) $\Omega(L)$

k mod m









miro

4/a

search(t: T[0..(m-1)]; key: k)

search(t: T[0..(m-1)]; key: k)	
i := key mod m	
e := t[i]	
e ≠ e→key = key	
true	*false*
return e	e := e→next
return ∅	

4/b

del(t: T[0..(n-1)]; p: T*)

p→prev ≠ ∅	
true	*false*
p→prev→next := p→next	i := p→key mod m
-	t[i] = ∅
p→next ≠ ∅	
true	*false*
p→next→prev := p→prev	skip
delete(p)	

4/c

Kitöltöttségi hányados: A hasító táblában tárolt adatok száma osztva a hasító tábla méretével

4/d

a-hoz) $mT(n) \in \Theta(1)$

$MT(n) \in \Theta(n)$

$AT(n) \in \Theta(n)$

$\frac{n}{m} = n \cdot \frac{1}{m} \in \Theta(n)$

b) $mT(n), MT(n),$

$AT(n) \in \Theta(1)$

miro